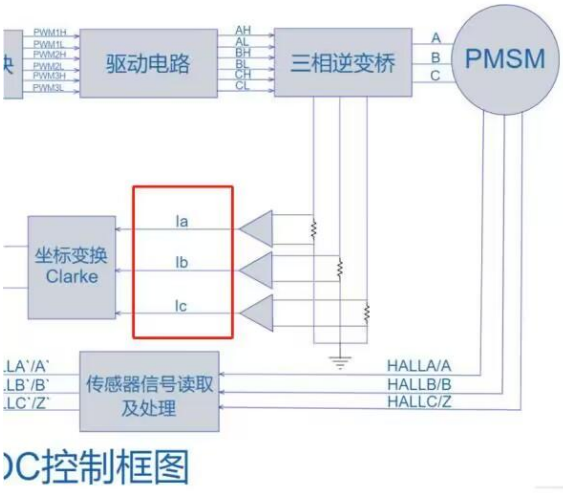
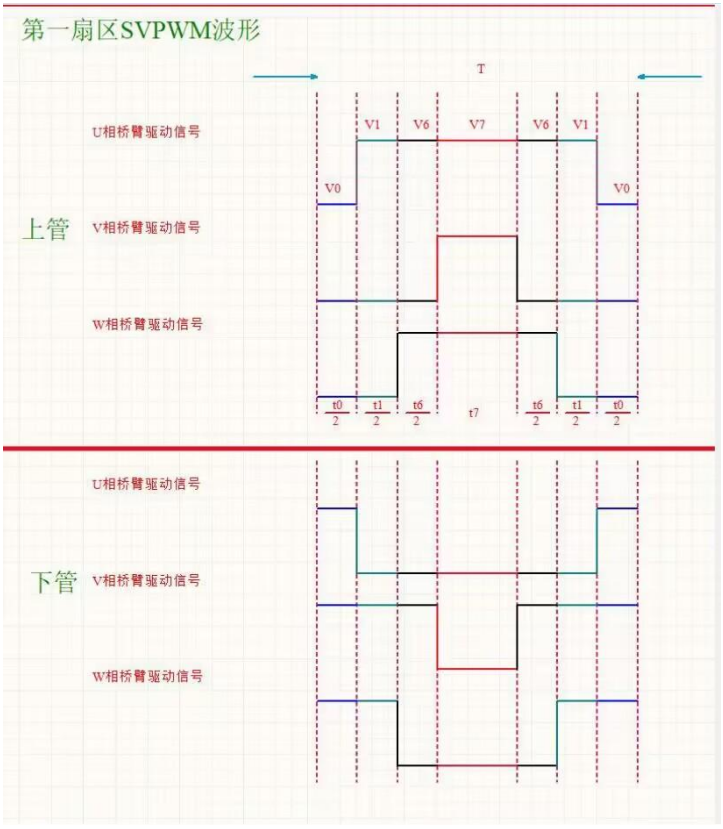
59-FOC 12

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。



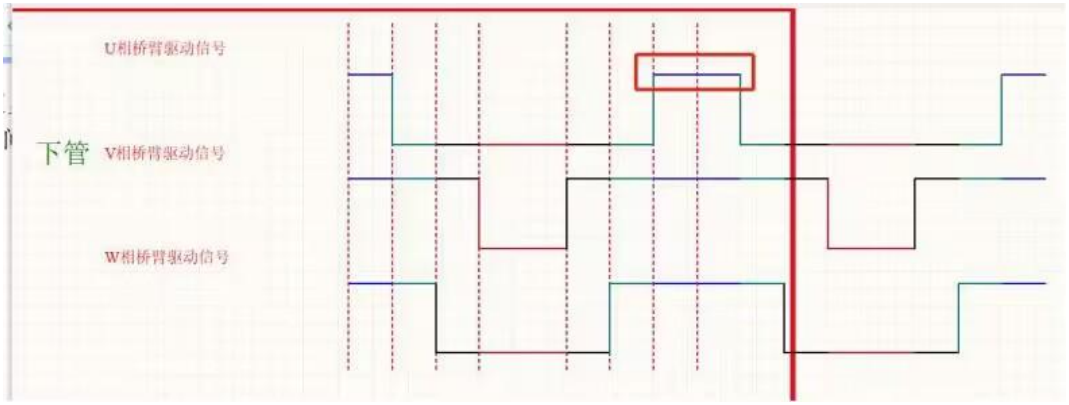
我们得到了框图中的 Ia，Ib，Ic，根据这三个值呢，去计算坐标变换，就可以得到 Iα 及 Iβ 。这时候，我们会碰到三相中，有其中一相的采样窗口比较小。这样的话，采样窗口小的采样值，可能会不准确。 那么，关于采样窗口的话，我们可以来看下 SVPWM 部分讲过的六个扇区的 PWM 波形。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家看下，这是我们之前绘制出来的第一扇区的波形。这里的采样点是设置在下桥开通的中点。

那么，从这幅图我们可以看出来，是不是 U 相下管的开通时间要小于 V 相跟 W 相的啊？



或者我多画一个周期，看起来会更明显一些。

问：上节课说的 Ia，Ib,Ic　就是根据电压的归一化来的是吗，电压当电流看　是吗，我再确认下？

答：是的，电压和电流是线性比例关系，程序只操作 Q15 格式的归一化电流，本质上只是电流更换了基本单位。

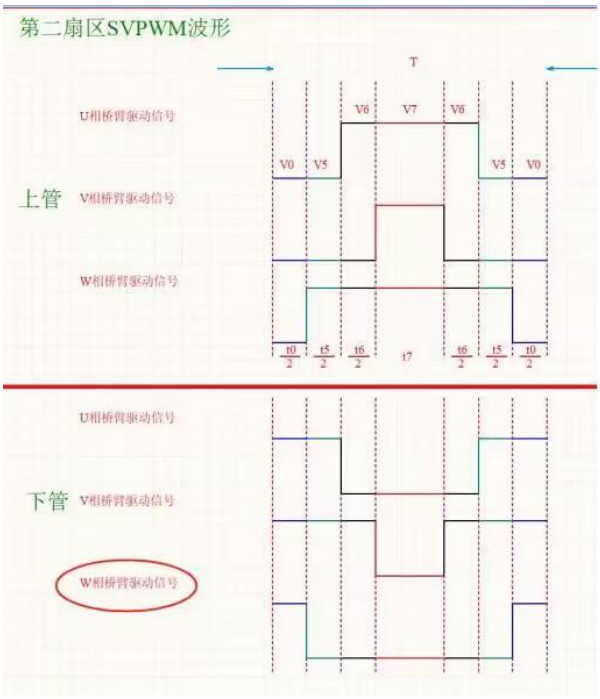
那么，在这个扇区内，会始终满足这个关系，U 相的 PWM 占空比最小，W 相次之，V 相最大。我们说的采样窗口，也就是三个下管导通的时间。导通的时间长，那么可用的采样窗口就越大，采样到的值就可能越准确。

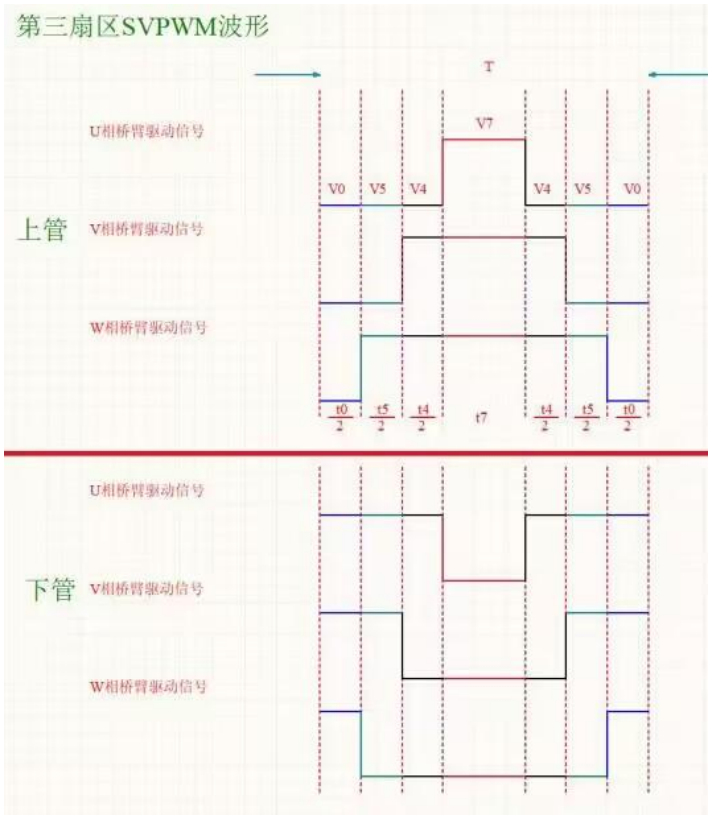
导通的时间短，则可用的采样窗口就越小，采样到的值就越不准确，很可能采样到干扰，因为图中的开关沿跳变实际上都有开关振荡。

当前面 U 相下管开通时间足够小时，这个采样窗口的电流信号会有开关振荡，这时就不适合把这个电流拿来做控制了。

再看一下第二扇区和第三扇区

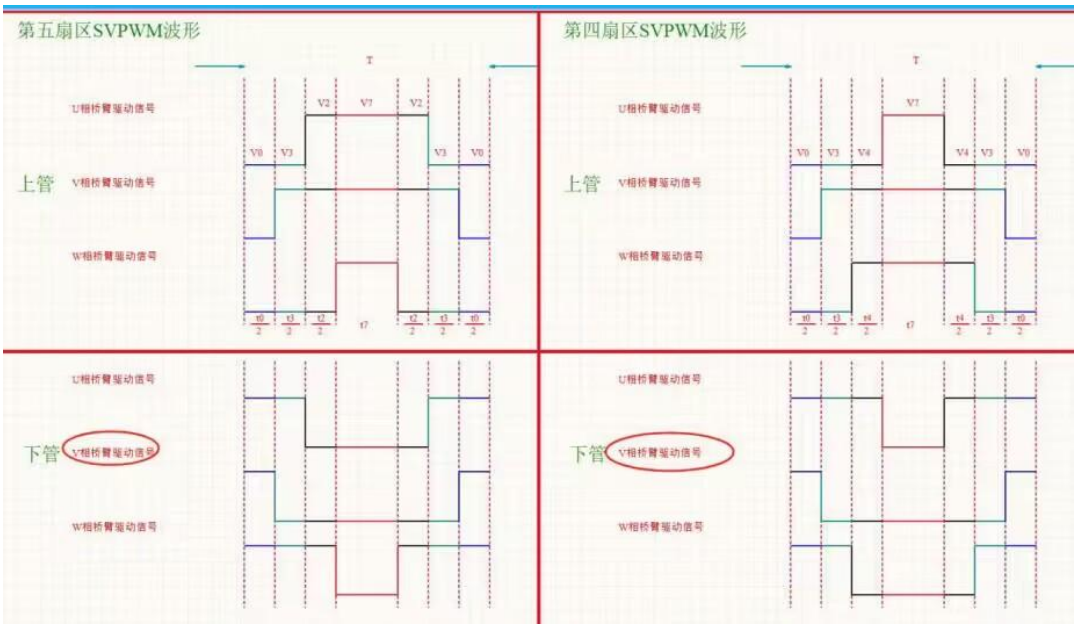
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



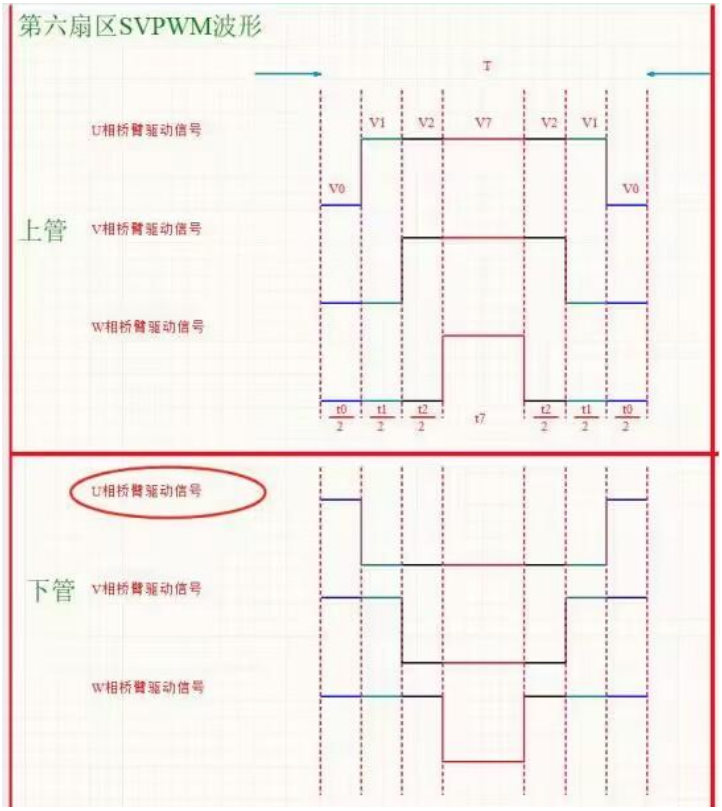


同理呢，对应的第二扇区，就是 W 相的采样窗口小，第三扇区，也是 W 相的采样窗口小。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



四五扇区呢，V 相的采样窗口小。



六扇区，也是 U 相的采样窗口小。

所以啊，结合六个扇区的 PWM 波形图，我们就可以知道，哪个扇区，对应的哪一相的采样窗口小。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这样的话，就能针对这个问题去解决。既然我们知道了哪个扇区的哪一相采样窗口小，我们就可以根据扇区再结合 Ia+Ib+Ic=0 的公式来求取采样窗口小的那一相相电流。

```
static void GetIabc(void)
{
    signed int sIa,sIb,sIc;

    sIa = (signed int)((tAdc.IoffsetA - tAdc.An0.ValueInstant) << 5);
    sIb = (signed int)((tAdc.IoffsetB - tAdc.An1.ValueInstant) << 5);
    sIc = (signed int)((tAdc.IoffsetC - tAdc.An2.ValueInstant) << 5);

    if((tUm.Sector == 1) || (tUm.Sector == 6))
        sIa = 0 - sIb - sIc;
    if((tUm.Sector == 4) || (tUm.Sector == 5))
        sIb = 0 - sIa - sIc;
    if((tUm.Sector == 2) || (tUm.Sector == 3))
        sIc = 0 - sIa - sIb;

    tIm.qIa = sIa;
    tIm.qIb = sIb;
    tIm.qIc = sIc;
}
```

此时呢，大家来看下这段代码，我们呢，就利用判断出来的扇区，然后结合扇区波形，来求取对应采样窗口小的相电流。

通过上面内容，大家应该能体会到三电阻采样的优缺点。

问：U 相对应 Ia　V 相对应 Ib　W 相对应 Ic 是吗？

答：是的。

```
static void GetIabc(void)
{
    signed int sIa,sIb,sIc;

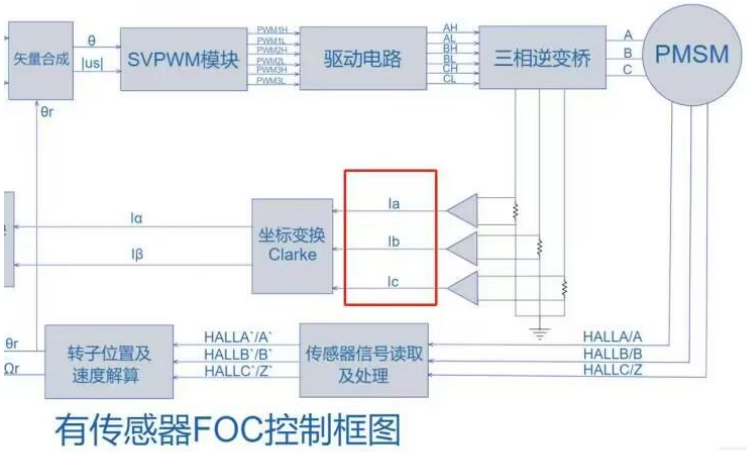
    sIa = (signed int)((tAdc.IoffsetA - tAdc.An0.ValueInstant) << 5);
    sIb = (signed int)((tAdc.IoffsetB - tAdc.An1.ValueInstant) << 5);
    sIc = (signed int)((tAdc.IoffsetC - tAdc.An2.ValueInstant) << 5);

    if((tUm.Sector == 1) || (tUm.Sector == 6))
        sIa = 0 - sIb - sIc;
    if((tUm.Sector == 4) || (tUm.Sector == 5))
        sIb = 0 - sIa - sIc;
    if((tUm.Sector == 2) || (tUm.Sector == 3))
        sIc = 0 - sIa - sIb;

    tIm.qIa = sIa;
    tIm.qIb = sIb;
    tIm.qIc = sIc;
}
```

所以呢，处理之后，就得到了实际要使用的三相相电流了，这三个值，就可以用于坐标变换了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

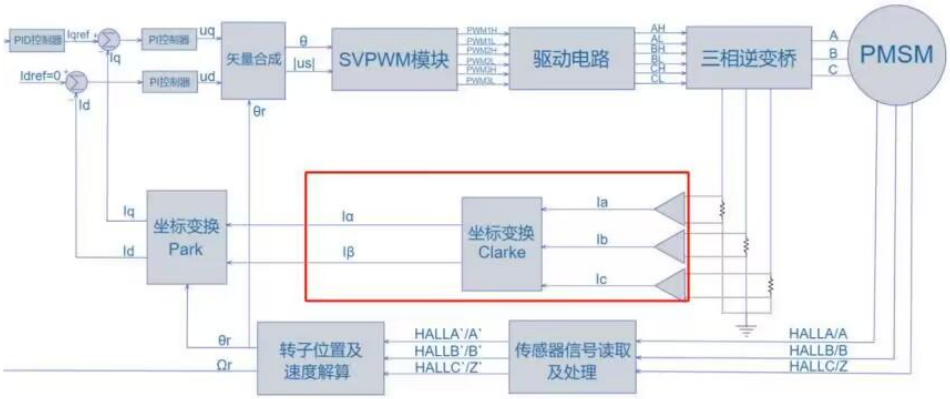


有传感器FOC控制框图

--- 本内容未经允许禁止商用转载，知识版权归©张飞电子所有

那么，再来看下这幅图，到现在为止呢，我们也就得到可用于坐标变换的三相相电流。

问：那在第二和第三扇区是 W 相开通时间最短，应该计算根据 KCL 计算 Ic 的电流吧？
答：是的。



有传感器FOC控制框图

Ia，Ib，Ic 都获取到之后，就可以通过 Clarke 变换来得到 Iα，Iβ。

$$I\alpha + jI\beta = Ia + j\left(\frac{1}{\sqrt{3}}Ia + \frac{2}{\sqrt{3}}Ib\right)$$

Clarke 变换的话，对应的公式我们之前推导出来了，就如上图所示。根据这个公式呢，我们需要写出对应的代码。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=============================================================================*/
//Clarke变换，计算公式
//Ialpha = Ia
//Ibeta = (Ia + 2Ib)/sqrt(3)
//Ibeta = 1/sqrt(3) * Ia + 1/sqrt(3) * 2Ib
/*=============================================================================*/
static void GetIalphabeta(void)
{
    UNION_S32 sTemp32A,sTemp32B;

    tIm.qIalpha = tIm.qIa; //Ialpha = Ia
    sTemp32A.S32 = __builtin_mulus(ONE_DIV_SQRT3,tIm.qIa);//1/sqrt(3) * Ia
    sTemp32B.S32 = __builtin_mulus(ONE_DIV_SQRT3,tIm.qIb);//1/sqrt(3) * Ib
    sTemp32B.S32 += sTemp32B.S32;//1/sqrt(3) * 2Ib
    sTemp32A.S32 += sTemp32B.S32;
    tIm.qIbeta = sTemp32A.Words.High;//Ibeta = 1/sqrt(3) * Ia + 1/sqrt(3) * 2Ib
}
```

大家看下对应的代码，对于 Iα 的话，直接用 Ia 来赋值。

```
/*=============================================================================*/
static void GetIalphabeta(void)
{
    UNION_S32 sTemp32A,sTemp32B;

    tIm.qIalpha = tIm.qIa; //Ialpha = Ia
    sTemp32A.S32 = __builtin_mulus(ONE_DIV_SQRT3,tIm.qIa);//1/sqrt(3) * Ia
    sTemp32B.S32 = __builtin_mulus(ONE_DIV_SQRT3,tIm.qIb);//1/sqrt(3) * Ib
    sTemp32B.S32 += sTemp32B.S32;//1/sqrt(3) * 2Ib
    sTemp32A.S32 += sTemp32B.S32;
    tIm.qIbeta = sTemp32A.Words.High;//Ibeta = 1/sqrt(3) * Ia + 1/sqrt(3) * 2Ib
}
```

然后，对于 Iβ 的话，需要进行一些转换

```
static void GetIalphabeta(void)
{
    UNION_S32 sTemp32A,sTemp32B;

    tIm.qIalpha = tIm.qIa; //Ialpha = Ia
    sTemp32A.S32 = __builtin_mulus(ONE_DIV_SQRT3,tIm.qIa);//1/sqrt(3) * Ia
    sTemp32B.S32 = __builtin_mulus(ONE_DIV_SQRT3,tIm.qIb);//1/sqrt(3) * Ib
    sTemp32B.S32 += sTemp32B.S32;//1/sqrt(3) * 2Ib
    sTemp32A.S32 += sTemp32B.S32;
    tIm.qIbeta = sTemp32A.Words.High;//Ibeta = 1/sqrt(3) * Ia + 1/sqrt(3) * 2Ib
}
```

转换的话，就根据公式来对应，涉及到乘法，而对应的是 Q 格式的乘法，所以我们要得到 Q15 结果的话，需要取乘法结果的高 16 位。然后呢，再根据公式，把对应的部分加起来。最终，也就得到了 Iβ 的值。

问：Ibeta 需要将 2 个 Q15 的数据相加，如何避免相加超出 1，也就是超前 Q15 的范围的情况呢？

答：这里在理论上就是等幅变换，也就是无论哪个量都在单位圆以内，不会大于 1。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================*/
//Clarke变换，计算公式
//Ialpha = Ia
//Ibeta = (Ia + 2Ib)/sqrt(3)
//Ibeta = 1/sqrt(3) * Ia + 1/sqrt(3) * 2Ib
/*==========================================================*/
static void GetIalphabeta(void)
{
    UNION_S32 sTemp32A,sTemp32B;

    tIm.qIalpha = tIm.qIa; //Ialpha = Ia
    sTemp32A.S32 = __builtin_mulus(ONE_DIV_SQRT3,tIm.qIa);//1/sqrt(3) * Ia
    sTemp32B.S32 = __builtin_mulus(ONE_DIV_SQRT3,tIm.qIb);//1/sqrt(3) * Ib
    sTemp32B.S32 += sTemp32B.S32;//1/sqrt(3) * 2Ib
    sTemp32A.S32 += sTemp32B.S32;
    tIm.qIbeta = sTemp32A.Words.High;//Ibeta = 1/sqrt(3) * Ia + 1/sqrt(3) * 2Ib
}
```


__builtin_mulus

说明：

计算乘积 *p0* x *p1*。函数参数为混合符号整型，函数结果为有符号长整型。可以使用命令行选项 -Wconversions 来检测意外的符号转换。该函数支持指令的所有寻址模式。

原型：

```
signed long __builtin_mulus(const unsigned int p0, const signed int p1);
```

参数：

p0	被乘数
p1	乘数

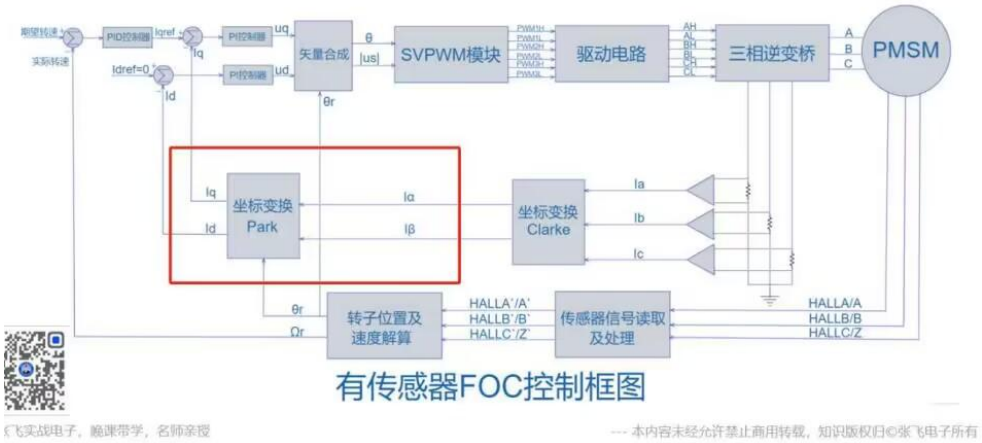
返回值：

返回乘积 *p0* x *p1* 的有符号长整型值。

汇编操作符 / 机器指令：

mul.us

无符号数乘有符号数，那下面我们就来分析 PARK 变换了。



飞实战电子，晚课带学，名师亲授
--- 本内容未经允许禁止商用转载，知识版权归©张飞电子所有

大家应该还记得 Park 变换对应的公式吧？

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

$$Id = I\alpha \cos\theta r + I\beta \sin\theta r$$

$$Iq = I\beta \cos\theta r - I\alpha \sin\theta r$$

$$\begin{bmatrix} Id \\ Iq \end{bmatrix} = \begin{bmatrix} \cos\theta r & \sin\theta r \\ -\sin\theta r & \cos\theta r \end{bmatrix} \begin{bmatrix} I\alpha \\ I\beta \end{bmatrix}$$

Park变换

大家看下，这是我们之前一起推导出来的 Park 变换对应的公式。要想得到 Id，Iq，需要 Iα，Iβ 以及 θr 的正弦值，余弦值。而这个 θr 的话，对应的转子的位置角。这个角度的话，之前我们是通过霍尔传感器计算出来的。所以这里呢，我们可以直接使用。我们只需要使用这个角度去查一次正弦表，就可以得到 sinθr 跟 cosθr。 而 Iα，Iβ 是上次课分析过的 Clarke 变换得到的。所以啊，我们要编写 Park 变换对应的代码的话，应该还是比较容易实现的。

```
/*=========================================================================*/
//Park变换
//Id = Ialpha * Cos θr + Ibeta * Sin θr
//Iq = -Ialpha * Sin θr + Ibeta * Cos θr
/*=========================================================================*/
static void GetIdq(void)
{
    tIm.qId = MulFrac(tIm.qIalpha, tCore.qCos) + MulFrac(tIm.qIbeta, tCore.qSin);
    tIm.qIq = MulFrac(-tIm.qIalpha, tCore.qSin) + MulFrac(tIm.qIbeta, tCore.qCos);
}
/*=========================================================================*/
```

编写出来呢，也就对应这个函数，只不过，这里的乘法呢，是两个 Q15 格式数据的乘法。这个乘法的实现，我们自己用汇编来写了。

那下面我们来分析下这个乘法的代码。

```
/*=========================================================
;函数入口：w0->X w1 -> Y X、Y是有符号数
;函数出口：w0 两个有符号数的乘积
=========================================================
_MulFrac:
    mov         w0,w4
    mov         w1,w5
    mpy         w4*w5,A
    sac.r       A,w0
    return
/*=========================================================
```

大家看下，实际上这里是使用了 mpy 这条乘法指令。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：`MulFrac(tIm.qIalpha,tCore.qCos)`，跟 `__builtin_mulus(ONE_DIV_SQRT3,tIm.qIa)`，都是乘法，有什么不同？

答：上面是小数乘法，下面是整数乘法。

表 3-11： DSP 类指令（dsPIC30F、dsPIC33F 和 dsPIC33E 器件）

汇编语法		说明	字数	周期数	页码
ADD	Acc	累加器相加	1	1	103
ADD	Wso,#Slit4,Acc	16 位有符号数与累加器相加	1	1(1)	104
CLR	Acc,[Wx],Wxd,[Wy],Wyd,AWB	清零累加器	1	1	186
ED	Wm*Wm,Acc,[Wx],[Wy],Wxd	求取欧几里德距离（无累加）	1	1	239
EDAC	Wm*Wm,Acc,[Wx],[Wy],Wxd	求取欧几里德距离	1	1	241
LAC	Wso,#Slit4,Acc	装载累加器	1	1(1)	265
MAC	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd,AWB	相乘并累加	1	1	275
MAC	Wm*Wm,Acc,[Wx],Wxd,[Wy],Wyd	平方并累加	1	1	277
MOVSAC	Acc,[Wx],Wxd,[Wy],Wyd,AWB	将 Wx 中内容传送到 Wxd，并将 Wy 中内容传送到 Wyd	1	1	293
MPY	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd	Wn 和 Wm 相乘，结果存入累加器	1	1	295
MPY	Wm*Wm,Acc,[Wx],Wxd,[Wy],Wyd	求平方，结果存入累加器	1	1	297
MPY.N	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd	Wn 和 Wm 相乘并取反，结果存入累加器	1	1	299
MSC	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd,	相乘并从累加器中减去乘积	1	1	301

这条乘法指令呢，属于 dsp 指令。这条乘法指令乘完的结果，会存放到累加器 A 中，然后我们使用的时候就可以从累加器中取出来。

语法： {标号:} MPY Wm * Wn, Acc {,[Wx],Wxd} {,[Wy],Wyd} {,[Wx] + = kx, Wxd} {,[Wy] + = ky, Wyd} {,[Wx] – = kx, Wxd} {,[Wy] – = ky, Wyd} {,[W9 + W12],Wxd} {,[W11 + W12],Wyd}

操作数： Wm * Wn ∈ [W4 * W5, W4 * W6, W4 * W7, W5 * W6, W5 * W7, W6 * W7]
Acc ∈ [A,B]
Wx ∈ [W8, W9]； kx ∈ [-6, -4, -2, 2, 4, 6]； Wxd ∈ [W4 ... W7]
Wy ∈ [W10, W11]； ky ∈ [-6, -4, -2, 2, 4, 6]； Wyd ∈ [W4 ... W7]
AWB ∈ [W13], [W13] + = 2

操作： (Wm) * (Wn) → Acc(A 或 B)
([Wx]) → Wxd； (Wx) + kx → Wx
([Wy]) → Wyd； (Wy) + ky → Wy

受影响的状态位： OA、OB、OAB、SA、SB 和 SAB

而我们查阅指令描述，有几个点我们要注意一下。

第一个，上图红框的部分，如果不使用的话，写指令的时候可以省略。

第二个，两个乘数对应的寄存器必须是按照蓝框中的组合来选择。

```
_MulFrac:
    mov     w0, w4
    mov     w1, w5
    mpy     w4*w5, A
    sac.r   A, w0
    return
/*=====================================
```

所以，这个时候大家来看，我们用了两条 mov 指令，分别把两个乘数送到 w4 跟 w5 寄存器中。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
_MulFrac:
    mov         w0, w4
    mov         w1, w5
    mpy         w4*w5, A
    sac.r       A, w0
    return
/*==============================
```

然后，直接用 mpy 指令，对两个乘数相乘，然结果呢，存到累加器 A 中。

```
_MulFrac:
    mov         w0, w4
    mov         w1, w5
    mpy         w4*w5, A
    sac.r       A, w0
    return
```

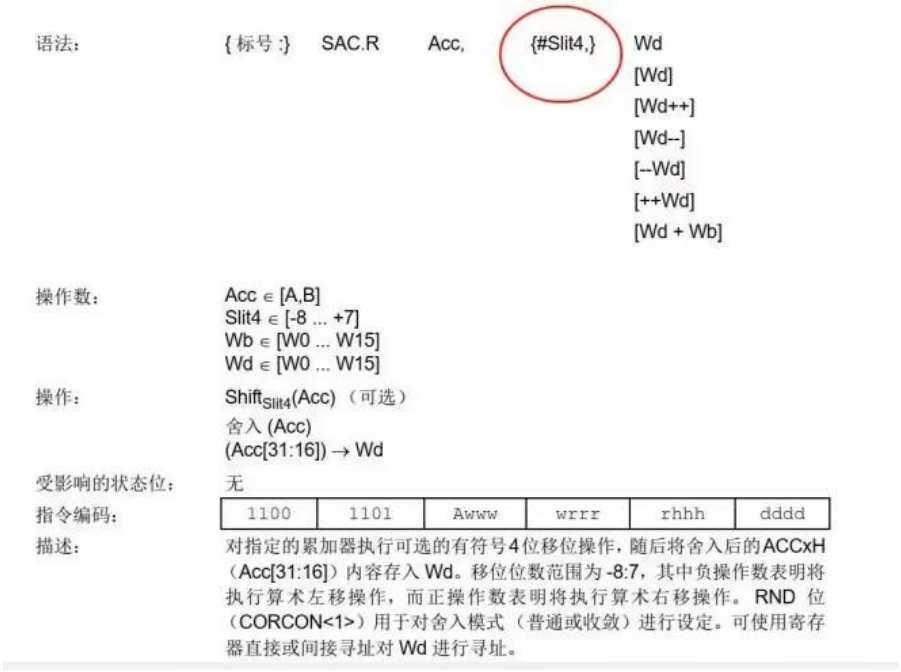
接着呢，使用了一条从累加器中取数据的指令，就把累加中的 bit16~bit31 取出来，存到 w0 中。

这样的话，也就实现了两个 Q15 相乘，得到的结果也是 Q15。

1	SAC	Acc,#Slit4,Wdo	保存累加器内容	1	1	389
	SAC.R	Acc,#Slit4,Wdo	保存舍入后的累加器内容	1	1	391

而这里使用的 SAC.R 指令，也是一条 DSP 指令。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



语法：{标号:} SAC.R Acc, {#Slit4,} Wd / [Wd] / [Wd++] / [Wd--] / [--Wd] / [++Wd] / [Wd + Wb]

操作数：Acc ∈ [A,B]
Slit4 ∈ [-8 ... +7]
Wb ∈ [W0 ... W15]
Wd ∈ [W0 ... W15]

操作：$Shift_{Slit4}$(Acc)（可选）
舍入 (Acc)
(Acc[31:16]) → Wd

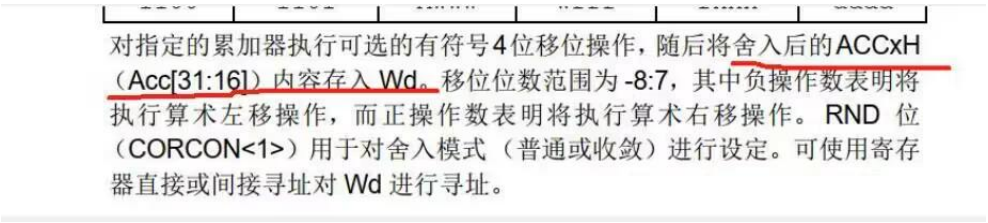
受影响的状态位：无

指令编码：

1100	1101	Awww	wrrr	rhhh	dddd

描述：对指定的累加器执行可选的有符号4位移位操作，随后将舍入后的ACCxH（Acc[31:16]）内容存入 Wd。移位位数范围为-8:7，其中负操作数表明将执行算术左移操作，而正操作数表明将执行算术右移操作。 RND 位（CORCON<1>）用于对舍入模式（普通或收敛）进行设定。可使用寄存器直接或间接寻址对 Wd 进行寻址。

同样呢，对于中间打括号的部分，也是可选的操作，如果我们不使用，就可以省略不写。



对指定的累加器执行可选的有符号4位移位操作，随后将舍入后的ACCxH（Acc[31:16]）内容存入 Wd。移位位数范围为 -8:7，其中负操作数表明将执行算术左移操作，而正操作数表明将执行算术右移操作。 RND 位（CORCON<1>）用于对舍入模式（普通或收敛）进行设定。可使用寄存器直接或间接寻址对 Wd 进行寻址。

这里描述了，这条指令的作用就是把累加器的 bit16~bit31 存到 wd 寄存器中。

```
_MulFrac:
    mov     w0, w4
    mov     w1, w5
    mpy     w4*w5, A
    sac.r   A, w0
    return
```

这就是这个乘法函数的分析。

问：默认是右移 4 位么？
答：默认是 0，就是不移位。

问：汇编语言编写的函数，输入输出参数，默认就是 W0，W1，顺序排列是么？

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

答：是的。

问：这里用到的数据多是 Q 格式的吧，所以都是整数的乘法吗？

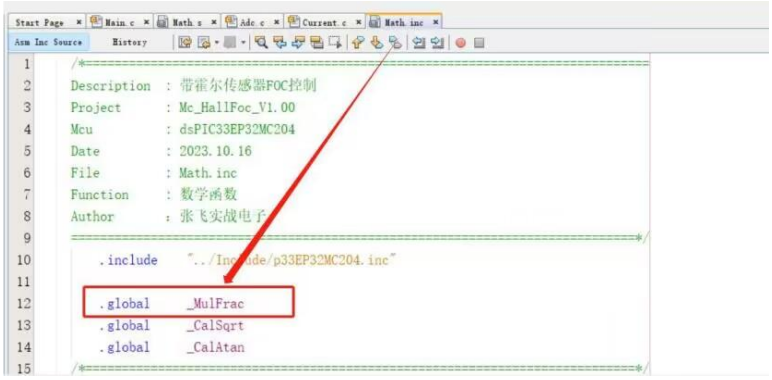
答：有的指令只能是整数，有的指令可以是小数，也可以整数，要看每条指令的详细解释。

表 3-11： DSP 类指令（dsPIC30F、dsPIC33F 和 dsPIC33E 器件）

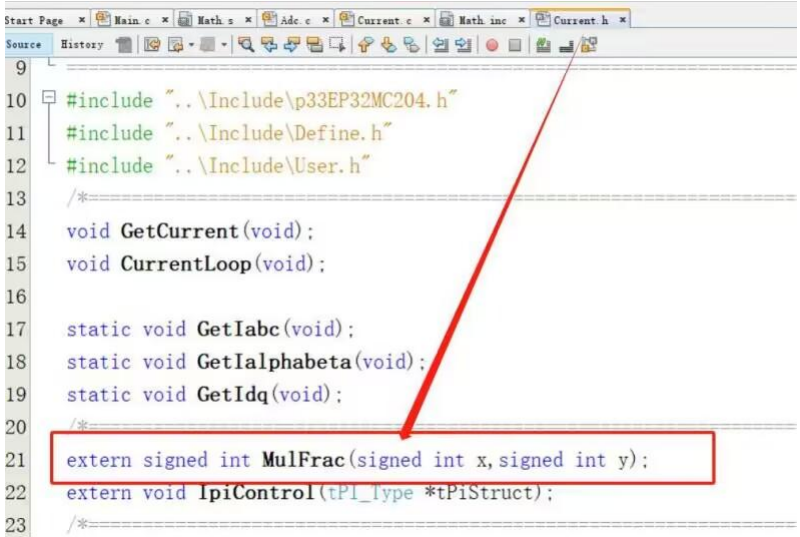
汇编语法		说明	字数	周期数	页码
ADD	Acc	累加器相加	1	1	103
ADD	Wso,#Slit4,Acc	16 位有符号数与累加器相加	1	1(1)	104
CLR	Acc,[Wx],Wxd,[Wy],Wyd,AWB	清零累加器	1	1	186
ED	Wm*Wm,Acc,[Wx],[Wy],Wxd	求取欧几里德距离（无累加）	1	1	239
EDAC	Wm*Wm,Acc,[Wx],[Wy],Wxd	求取欧几里德距离	1	1	241
LAC	Wso,#Slit4,Acc	装载累加器	1	1(1)	265
MAC	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd,AWB	相乘并累加	1	1	275
MAC	Wm*Wm,Acc,[Wx],Wxd,[Wy],Wyd	平方并累加	1	1	277
MOVSAC	Acc,[Wx],Wxd,[Wy],Wyd,AWB	将 Wx 中内容传送到 Wxd，并将 Wy 中内容传送到 Wyd	1	1	293
MPY	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd	Wn 和 Wm 相乘，结果存入累加器	1	1	295
MPY	Wm*Wm,Acc,[Wx],Wxd,[Wy],Wyd	求平方，结果存入累加器	1	1	297
MPY.N	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd	Wn 和 Wm 相乘并取反，结果存入累加器	1	1	299
MSC	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd,AWB	相乘并从累加器中减去乘积	1	1	301
NEG	Acc	对累加器内容求补	1	1	335
SAC	Acc,#Slit4,Wdo	保存累加器内容	1	1	389
SAC.R	Acc,#Slit4,Wdo	保存舍入后的累加器内容	1	1	391
SFTAC	Acc,#Slit6	算术移位累加器内容 Slit6 位	1	1	397
SFTAC	Acc,Wn	算术移位累加器内容 (Wn) 位	1	1	398
SUB	Acc	累加器相减	1	1	410

注 1： 在 dsPIC33E 和 PIC24E 器件中，对非 CPU 特殊功能寄存器的读操作和读 - 修改 - 写操作，都需要 1 个额外周期 —— 与 dsPIC30F、dsPIC33F、PIC24F 和 PIC24H 器件相比。

这些 DSP 指令功能很强大，只要 1 个机器周期数时间。



接着呢，我们编写的汇编函数，需要在对应的头文件中去声明。



本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

然后呢，再到要使用的 C 文件中去声明，最后呢，就可以正常按照 C 函数的调用方式来调用了。

刚才那个 mpy 汇编指令，大家注意啊，十分强大，可以做整数乘法，还可以做有符号小数乘法。

寄存器 3-2： CORCON：内核控制寄存器（续）

bit 3 **IPL3**：CPU 中断优先级状态位 3(3)
1 = CPU 中断优先级大于 7
0 = CPU 中断优先级等于或小于 7

bit 2 **SFA**：堆栈帧有效状态位
1 = 堆栈帧有效。无论 DSRPAG 和 DSWPAG 值如何，W14 和 W15 都寻址 0x0000 至 0xFFFF
0 = 堆栈帧无效。W14 和 W15 寻址 EDS 或基本数据空间

bit 1 **RND**：舍入模式选择位(1)
1 = 使能有偏（常规）舍入
0 = 使能无偏（收敛）舍入

bit 0 **IF**：整数或小数乘法器模式选择位(1)
1 = 使能整数模式的 DSP 乘法运算
0 = 使能小数模式的 DSP 乘法运算

注 1：该位仅在 dsPIC33EPXXXMC20X/50X 和 dsPIC33EPXXXGP50X 器件上可用。
2：该位将总是读为 0。
3：IPL3 位与 IPL<2:0> 位（SR<7:5>）组合形成 CPU 中断优先级。

默认是小数乘法。

寄存器 3-2： CORCON：内核控制寄存器

R/W-0	U-0	R/W-0	R/W-0	R/W-0	R-0	R-0	R-0
VAR	—	US<1:0>(1)		EDT(1,2)	DL<2:0>(1)		
bit 15							bit 8

R/W-0	R/W-0	R/W-1	R/W-0	R/C-0	R-0	R/W-0	R/W-0
SATA(1)	SATB(1)	SATDW(1)	ACCSAT(1)	IPL3(3)	SFA	RND(1)	IF(1)
bit 7							bit 0

图注：
R = 可读位 W = 可写位 U = 未实现位，读为 0
-n = POR 时的值 1 = 置 1 0 = 清零 x = 未知
C = 可清零位

bit 15 **VAR**：可变异常处理延时控制位
1 = 使能可变异常处理延时
0 = 使能固定异常处理延时

bit 14 **未实现**：读为 0

bit 13-12 **US<1:0>**：DSP 乘法无符号 / 有符号控制位(1)
11 = 保留
10 = DSP 引擎执行混合符号乘法运算
01 = DSP 引擎执行无符号乘法运算
00 = DSP 引擎执行有符号乘法运算

bit 11 **EDT**：DO 循环提前终止控制位(1,2)
1 = 在当前循环迭代结束时终止执行 DO 循环
0 = 无影响

关键是所有 DSP 指令只要 1 个机器周期。

问：这是这颗芯片价格贵的主要原因么？
答：不是，是中美关系和芯片的年龄较大的原因。

问： 默认的小数乘法配置需要改吗？
答：课程中没有改。

```
/*=====================================================================*/
//Park变换
//Id = Ialpha * Cos0r + Ibeta * Sin0r
//Iq = -Ialpha * Sin0r + Ibeta * Cos0r
/*=====================================================================*/
static void GetIdq(void)
{
    tIm.qId = MulFrac(tIm.qIalpha, tCore.qCos) + MulFrac(tIm.qIbeta, tCore.qSin);
    tIm.qIq = MulFrac(-tIm.qIalpha, tCore.qSin) + MulFrac(tIm.qIbeta, tCore.qCos);
}
/*=====================================================================*/
```

那么这样的话，这个 Pakr 变换对应的代码，就实现了，也就是说，现在可以得到 Id，Iq 了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

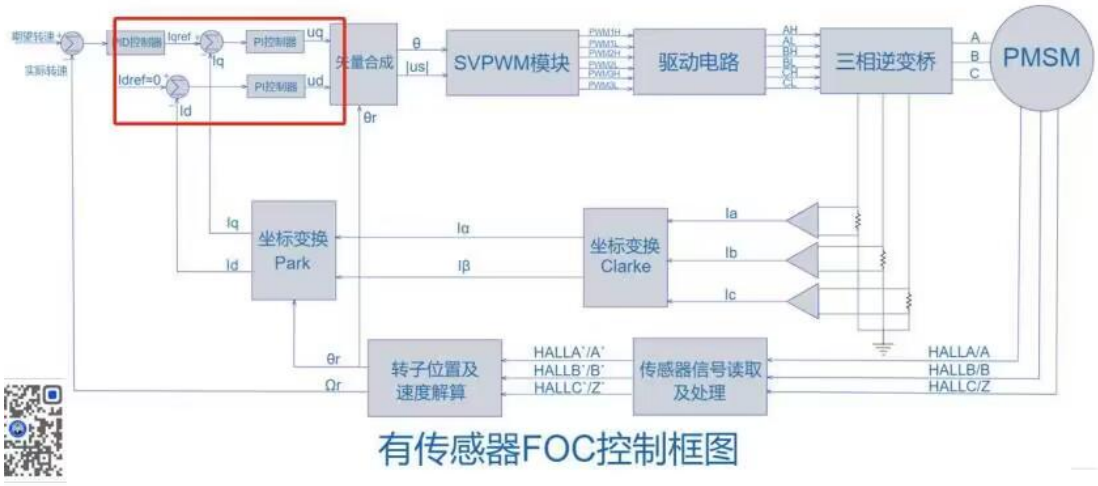
例 13-1：　　函数调用模型

```
void
params0(short p0, long p1, int p2, char p3, float p4, void *p5)
{
        /*
        ** W0          p0
        ** W1          p2
        ** W3:W2       p1
        ** W4          p3
        ** W5          p5
        ** W7:W6       p4
        */
        ...
}
```

13.8.2　返回值

对于 8 位或 16 位标量，函数返回值在 W0 中返回；对于 32 位标量为 W1:W0；对于 64 位标量为 W3:W2:W1:W0。聚合通过 W0 间接返回，它由函数调用方设置为包含聚合值的地址。

汇编有默认的形参和返回值，上面大家可以看一下。然后函数的返回值呢，按照这个原则来确定。因为我们是返回一个 16 位的数据，所以返回值就对应 W0 寄存器的值。



有传感器FOC控制框图

而得到了 Id，Iq 之后，我们要对这个电流进行控制。要实现电流闭环控制的话，这里就需要 PI 控制。

$$U(n) = Kp*error(n) + Ki* \sum_{i=0}^{n} Kp*error(i)$$

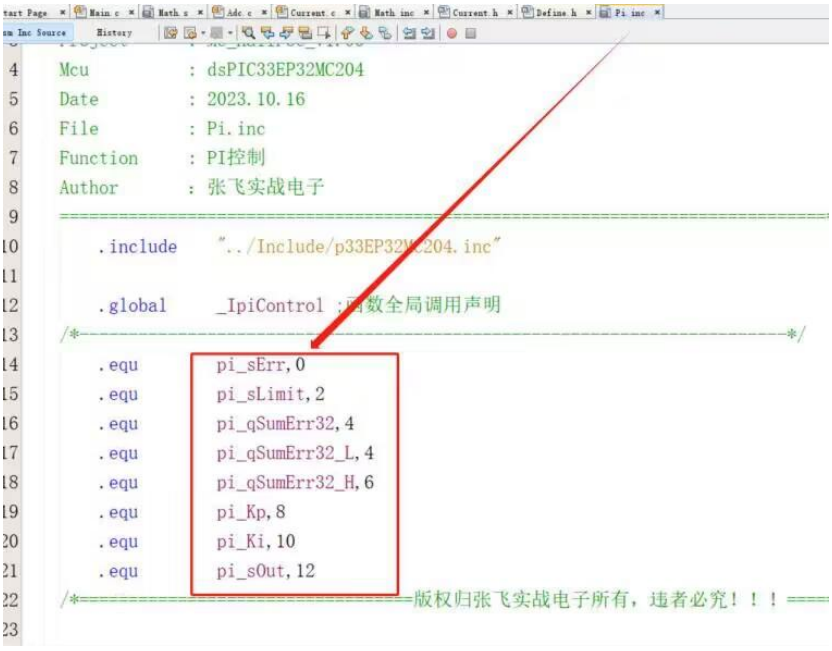
之前我们讲 PI 控制的理论的时候，讲过这个离散化的 PI 控制器公式。写代码的话，也就是要把这个离散化的公式在单片机中实现。这里呢，我们先用汇编实现一下，后面速度环控制的 PID，我们再用 C 也实现一遍。那么，要实现这个公式的话，需要先做一些变量定义。

问：编译器，在 C 和汇编互调，是否也遵守类似 ATPCS 规范吗？
答：不同的编译系统的规则一般会不一样，具体要看使用手册。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================
typedef struct
{
    signed int sErr;//误差
    signed int sLimit;//PI控制输出限幅
    UNION_S32 qSumErr32;//积分项累加值
    unsigned int Kp;//比例项系数
    unsigned int Ki;//积分项系数
    signed int sOut;//PID控制输出
}tPI_Type;
/*==================================版权归张飞实
```

大家看下，我们定义了一个结构体，里面成员分别是：误差，输出限幅值，积分累加值，比例系数，积分系数以及控制器的输出值。这些量呢，我们后面会使用到。



```
4   Mcu          : dsPIC33EP32MC204
5   Date         : 2023.10.16
6   File         : Pi.inc
7   Function     : PI控制
8   Author       : 张飞实战电子
9   ================================================================================
10      .include    "../Include/p33EP32MC204.inc"
11
12      .global     _IpiControl ;函数全局调用声明
13  /*--------------------------------------------------------------------------------*/
14      .equ        pi_sErr,0
15      .equ        pi_sLimit,2
16      .equ        pi_qSumErr32,4
17      .equ        pi_qSumErr32_L,4
18      .equ        pi_qSumErr32_H,6
19      .equ        pi_Kp,8
20      .equ        pi_Ki,10
21      .equ        pi_sOut,12
22  /*==========================版权归张飞实战电子所有，违者必究！！！====
23
```

然后，因为这个 PI 控制函数，会采用汇编语言来实现。所以，C 语言定义的结构体中的成员变量，我们需要在汇编中也要能访问。而变量的本质，实际上就是内存中的一段地址空间。例如，1 个 16 位的变量，就对应内存中一个 16 位的地址空间， 所以，汇编中的变量，其实就是地址。我们访问某个地址，其实就相当于 C 语言中访问某个变量，它们是对等的。

问： 这里的. 也表示当前地址？

答：. equ 表示汇编伪指令。

问： 这个不涉及到结构体成员地址对齐？

答：结构成员是顺序排列的，这里要注意的是我们正常定义变量要按偶数长度往下定义就可以了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================
typedef struct
{
    signed int sErr;//误差
    signed int sLimit;//PI控制输出限幅
    UNION_S32 qSumErr32;//积分项累加值
    unsigned int Kp;//比例项系数
    unsigned int Ki;//积分项系数
    signed int sOut;//PID控制输出
}tPI_Type;
/*==========================版权归张飞实占
```

而对于这个结构体的变量来说，它会分配到一个基地址。然后，访问各个成员变量访问的话，就是基地址+偏移地址。例如，这个 sErr 的偏移地址就是 0，sLimit 的偏移地址就是 2。

问：最小存储单元是 8，咱们 16 位寄存器，所以一次移动两个？

答：这个单片机是 16 位单片机，这里的 int 是 16 位。是的，这里说 RAM 存储空间，不说寄存器。

而 qSumErr32 是 32 位的，它的偏移地址是 4，但是它占两个地址单元，所以 Kp 的偏移地址就是 8。

```
/*----------------------------------------------------------
.equ        pi_sErr,0
.equ        pi_sLimit,2
.equ        pi_qSumErr32,4
.equ        pi_qSumErr32_L,4
.equ        pi_qSumErr32_H,6
.equ        pi_Kp,8
.equ        pi_Ki,10
.equ        pi_sOut,12
/*==========================版权归张飞实战电
```

对于这个汇编语言中的符号定义，就是根据这些成员变量的偏移地址来定义的。

今天内容就到这里，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。